RESEARCH SOFTWARE / LANGUAGE DESIGN

Mathematical Intent, Formal Evidence

A proof-language design informed by ProveIt and Leant

A proposal for a Mathematical Proof Layer over Lean

September 5, 2026

Abstract

A concise formal proof should record the mathematical decisions that make an argument work, rather than the accidental arrangement of a prover's library interfaces. It should not, however, make the hypotheses that justify those decisions disappear. This article proposes MPL, a *Mathematical Proof Layer* over Lean, organized around typed mathematical actions and explicit proof contracts. A sentence such as "differentiate the induction hypothesis locally on the open set" would generate a precise collection of obligations and a proof-term constructor, not invoke an unchecked interpretation of mathematical prose.

The proposal is grounded in four source-level case studies from ProveIt: Abel polynomial coefficients, iterated derivatives of an autonomous equation, an associated Stirling recurrence, and a weighted scaling identity. Leant supplies a complementary design lesson: structural synthesis, tactic suggestions, ranking, and behavioral evidence must remain separated from the authority to accept a theorem. We develop a surface language, a scoped intermediate representation, a small elaboration calculus, a relative-soundness argument, an origin-bound synthesis protocol, and an incremental implementation and evaluation plan. Particular attention is paid to theorem-statement fidelity, dependent witnesses, mathematical domains, boundary cases, and reproducible checking. The proposed gain is not merely fewer characters: it is a better correspondence between an author's mathematical explanation and the evidence a machine checks.

Status. This is a design article based on static source inspection, not a report of an implemented MPL compiler. The mathematical examples are reconstructed and explained; the proposed syntax and compiler rules have not been executed in Lean. No compression ratio, speedup, or user-study outcome is claimed.

Contents

1 The problem is not simply that Lean has too many words

A mathematician may explain a coefficient identity in three moves: apply Lagrange inversion, combine exponentials, and read off a coefficient. A formal proof can spend considerably more space establishing that substitution is permitted, converting integers into a coefficient algebra, rearranging products, and relating two encodings of a generating function. These are not all the same kind of work. Some express essential mathematics; others are consequences of the chosen representation; still others merely connect library interfaces.

The central design question is therefore not "How can every proof be shortened?" It is:

> *How can the author state the mathematical plan at an appropriate level of abstraction, while the system reconstructs and exposes all the evidence needed to make that plan valid?*

A one-line call to an unrestricted search procedure can be short but uninformative. A long proof can be clear and valuable. Conversely, a pleasant paragraph can conceal a missing hypothesis. The language should optimize for *faithful, inspectable mathematical compression*, rather than treating source length as the only objective.

Lean already distinguishes surface syntax, elaboration, and kernel-level terms, and already supports structured proof commands and extensible automation [8]. MPL should build on that architecture rather than introduce another foundational logic. Its proposed contribution is a coherent interface between mathematical actions, domain-aware elaboration, scoped synthesis requests, and replayable proof artifacts.

1.1 What was reviewed, and what was not

The ProveIt sample consists of four files at revision

`24ce8bd743eaab64a91ce90725ea00f498d319d2`.

The inspected declarations are identified in Table 1 and Appendix B. They were selected to expose different kinds of proof engineering, not to estimate a repository-wide average. The review does not establish that these proofs are minimal, and it does not constitute a build or axiom audit of the repository.

Leant's synthesis and interactive-proving documentation was inspected, including its exact-goal verification policy and its warning that a type may admit behaviorally inappropriate terms. Its semantic-origin and behavioral-selection boundaries were also examined. The final README and public selection-wrapper references use revision

`3a40904be8a410d832d9ab6900b3d3e7b425eccb`.

The earlier detailed internals inspection uses the separately recorded revision `c36adf1...`; the distinction is retained rather than implying that every implementation module was audited at the later revision [5, 6, 7].

1.2 Four kinds of detail

Strategic detail identifies the proof's organizing idea: use an inverse-series theorem, strengthen an induction hypothesis, or pass to a generating function. It should normally remain in the readable proof.

Sample	Mathematical plan	Main elaboration pressure
Abel coefficients	Lagrange inversion and exponential coefficients	Substitution, scalar embeddings, factorial normalization
Autonomous derivatives	Induction and the chain rule	Local equality, neighborhood reasoning, range-local hypotheses
Associated Stirling recurrence	Differentiate an exponential generating function	Coefficient shifts, natural-number boundaries, transport through $\mathbb{Q}$
Weighted scaling	Coordinatewise algebraic verification	Finite extensionality and nonzero denominators; an already compact control

Table 1: A purposive sample of proof patterns, not a quantitative benchmark.

Semantic detail determines whether the argument is valid: an open domain, a nonzero scalar, a permissible substitution, a faithful embedding, or an index restriction. Its proof may be automatic, but its existence must be visible and its status must never be guessed.

Representational detail arises from implementing mathematics: a natural number viewed in a rational algebra, a sequence viewed as an exponential generating function, or a function equality expressed using extensionality. This is the best target for reusable elaboration rules.

Incidental detail comes from a particular library or tactic interface: the order of rewrite calls, the spelling of an internal lemma, or an association of multiplications required by a matcher. It should usually be hidden behind a stable mathematical interface.

The boundaries are contextual. A change of representation can be the central idea of one proof and routine in another. MPL therefore needs expandable explanations, not a permanent classification declaring certain reasoning intrinsically unimportant.

2 What Leant contributes to the design

2.1 Type-directed synthesis belongs below the mathematical narrative

Leant asks for a type and constructs candidate inhabitants. Its documented fragment includes structural logical connectives and selected inductive and polymorphic constructions; candidate terms are checked again by Lean against the requested goal. Its interactive proving mode also suggests tactic steps and can identify suggestions that close a goal [5].

This is particularly well suited to *proof plumbing*. Given

$$f : B \to C, \qquad g : A \to B, \qquad x : A,$$

a synthesizer can construct $f(g(x))$ without requiring the author to spell out the applications. Similarly, it can assemble conjunctions, specialize quantified hypotheses, and connect a supplied local fact to a theorem's premise. A mathematical language should make this work a service, rather than turning every missing application into a user-visible hole.

This observation does not imply that the present Leant engines understand formal power series, topology, or arbitrary dependent mathematics. The proposed architecture routes a goal to a synthesizer only after a capability check. Domain-specific reasoning needs separate certified rules, suitable theorem retrieval, or an explicit argument from the author.

2.2 An inhabitant is not necessarily the intended construction

The README gives an important counterexample to a simplistic synthesis story. A state-transformer type can admit both a term that threads the updated state and a term that reuses the original state. Likewise, the type of an option-binding operation admits a constant failure result [5]. All can be type-correct.

For MPL, this means that automatic *proof* completion and automatic *object* selection must have different user-interface policies. A proof of an already fixed proposition can often be chosen without bothering the author. A function, an inverse branch, a normalization, or a witness that later computations use may require a stronger specification or explicit acceptance. A shorter inhabitant must not silently redefine what the mathematics is about.

2.3 Evidence must remain attached to the exact problem

Leant's internals describe a semantic-origin record tying source and search goals to provider bindings, rendering information, and completeness information. The behavioral path retains exact candidate/problem associations. Its public selection wrapper explicitly separates a validated partition of accepted candidates from the independent question of whether their behavior is established [6, 7].

The transferable principle is stronger than "run a checker at the end." A result must be bound to the exact goal, context, candidate, interpretation, and assumptions for which it was obtained. A useful MPL proof artifact should carry these associations throughout elaboration. A term's printed spelling is not a substitute for its typed identity, and a digest is not a substitute for proof.

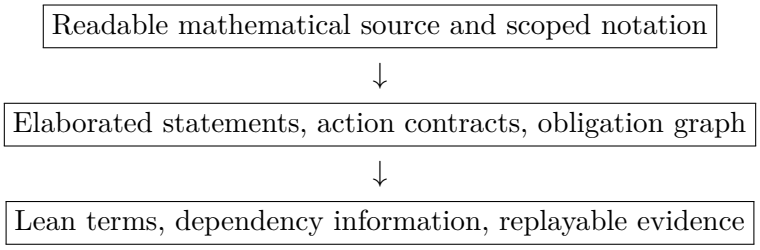
2.4 Failure, refutation, and bounded evidence are different outcomes

Leant distinguishes a complete, lossless fragment's negative result from a bounded search that found no term. Its behavioral documentation also distinguishes a model-relative counterexample receipt from a concrete Lean proof [5, 6]. MPL should preserve these distinctions, not collapse them into a green or red badge.

For example, an external search procedure returning "no candidate within budget" supplies no proof of $\neg P$. Even an exhaustive result in an abstraction does not establish a statement about the original goal without an appropriate translation theorem. Conversely, a candidate that passes a bounded behavioral test is not thereby proved to meet an unbounded specification. The default response to unsupported translation, exhausted search, or unvalidated solver output is an outstanding obligation.

3 The proposal in one view

MPL is a proposed *Mathematical Proof Layer* over Lean. The name is descriptive and provisional. The language has three connected views:



The author ordinarily works in the first view. The second explains what the system understood and what remains to be justified. The third provides the final verification artifact. No arrow permits a heuristic to add an assumption or accept an unproved claim.

3.1 A mathematical action is not an unchecked English instruction

Consider the proposed sentence:

```
differentiate IH locally on U using openness, chain_rule.
```

It has a limited grammar. The label `IH` resolves to a particular fact; `U` resolves to a particular set; `locally` selects a neighborhood-based rule. A method adapter computes a precise contract, including the equality to differentiate, the relevant point and neighborhood, and the derivative facts needed by the chain rule. The adapter returns a proof-term construction with holes for these premises. Only checked solutions of those holes complete the step.

A completely free-form sentence may be accepted as an *editing request*: an assistant can propose an MPL block. It should not directly acquire proof authority. The resulting block still goes through deterministic syntax, formula elaboration, visible statement resolution, and kernel checking.

3.2 A small surface vocabulary, extensible mathematical methods

The foundational vocabulary should be modest: declarations, local assumptions, named facts, reductions, witnesses, case splits, induction, calculations, and explicit conclusion. More specialized phrases, such as coefficient extraction or differentiation, belong to versioned method libraries. This avoids embedding the whole of mathematics into a special-purpose parser.

A typical proof could look like this:

```
proof
  let B := the formal exponential tail after degree s.
  have column: C(k) = B^k / k! by Bell_column.
  have derivative: D(B) = B + z^s / s! by tail_derivative.
  differentiate column at k + 1 using derivative.
  normalize to D(C(k+1)) = (k+1)*C(k+1) + z^s/s!*C(k).
  compare exponential coefficients at n, respecting support.
  conclude by faithful transport from Q to Nat.
end
```

Listing 1: Proposed syntax; method names denote specified interfaces, not existing commands.

The surrounding theorem fixes the meaning of B, C, s, n, k, the coefficient ring, and the target equation. Section 8 makes this example precise. In particular, the final line does *not* posit a ring homomorphism from $\mathbb{Q}$ to $\mathbb{N}$: it uses injectivity of the natural-number embedding into $\mathbb{Q}$ to reflect an equality.

3.3 Defaults are policies, not logical principles

A project may choose a default algebraic normalizer, a local theorem-search budget, or a conventional notation for coefficients. These settings affect convenience and performance. They cannot turn a failed obligation into a theorem. A method which succeeds under one policy and times out under another has changed its operational behavior, not the meaning of its target.

For reproducibility, the chosen settings and method versions are recorded with an accepted proof. In particular, the phrase "by routine algebra" must name a bounded, inspectable service configuration; it is not an axiom that whatever the author considers routine is true.

4 Surface language and mathematical scope

4.1 Declarations should resemble mathematics without hiding their types

A context for the Abel coefficient theorem might be written as follows:

```
context
  A is a commutative Q-algebra.
  a, x belong to A.
  T is a formal power series over A.
  assume equation: T = z * exp(-a*T).
end
```

The formal-power-series notation pack must define exactly how `exp(-a*T)` elaborates. Here it denotes substitution of the formal exponential with parameter $-a$ into T, not analytic exponentiation of a function and not a new primitive exponential on an arbitrary ring. The resolved statement remains available in the semantic view.

An omitted binder may be inferred from an explicit surrounding declaration. It should not be invented as an additional theorem parameter simply because a name is unrecognized. For theorem statements, the default should be strict undeclared-name checking, analogous in spirit to the source files' use of `autoImplicit false` [1, 3].

4.2 Named facts and backward reasoning

The commands

```
have h: Q from facts using method.
suffices Q by bridge.
conclude P from h, other_facts.
```

have different meanings. The first adds $h : Q$ only after a proof of Q has been obtained. The second changes the working goal to Q while recording an obligation for a bridge $Q \to P$, where P was the original goal. The last closes the current goal with a term of its exact type. Reversing the bridge to $P \to Q$ would be a logical error, even if the prose sounded plausible.

A label is a scoped identifier, not a search keyword. Pronouns such as "this" may refer only to an unambiguous designated fact; otherwise the interface asks for a label. The proof remains a document rather than an unpredictable conversation with an unbounded context.

4.3 Induction should expose the invariant, not the recursor plumbing

The useful high-level operation is not simply `induct n`. It is "prove this invariant for all n, keeping these parameters arbitrary." A proposed syntax is:

```
induct on n with invariant P(n), generalizing x.
  base: ...
  step n, assuming IH: forall x, P(n, x): ...
end
```

The invariant and generalized variables are part of the semantic contract. The compiler must check the base and step against the selected induction principle. It must not strengthen or weaken the invariant invisibly in order to make the search succeed. A suggested stronger invariant can be valuable, but it appears as a proposed edit.

Structural and well-founded induction need different adapters. In the latter, the relation and its well-foundedness evidence are explicit dependencies. A recursive proof cannot disguise a circular invocation as an induction step.

4.4 Equality, equivalence, and representation changes are distinct

Mathematicians routinely move between a function and its graph, a sequence and its generating function, or natural numbers and their rational images. A language that identifies all these views by mere notation is dangerous. MPL should instead make representation changes proof-producing.

For an equality target $u = v$, a command `work in B via f` requires a suitable equality-reflection theorem, typically injectivity of $f : A \to B$. For an implication, one may need only a one-way preservation lemma. For an order comparison, injectivity alone is not enough: an order-reflection property may be needed. All three should have different contracts.

Automatic transport may infer a canonical map already fixed by the context. It may not select an arbitrary isomorphism that changes the interpretation of the statement. The accepted artifact records which map was used, the direction of reasoning, and the relevant preservation or reflection lemma.

4.5 Calculations and implicit arguments

A calculation is a chain of explicitly typed relations:

```
calculate
  expression_0
    = expression_1 by substitution.
    = expression_2 by exponential_algebra.
    = expression_3 by coefficient_formula.
end
```

Each edge produces a proof of the displayed relation; transitivity composes the edges. Mixed equality and inequality chains need declared compatibility rules. Omitted intermediate expressions may be synthesized during exploration, but the accepted version should retain the mathematical turning points that explain the argument.

Implicit arguments are solved by ordinary elaboration first. More ambitious inference can fill theorem arguments, assemble local proof terms, and select registered representation lemmas. It must not equate distinct mathematical objects merely because both would make the final goal provable.

4.6 An escape hatch is necessary

A block of ordinary Lean should remain available for exceptional steps. It receives the exact elaborated local context and target and returns a term subject to the same verification policy. This allows incremental adoption and prevents the surface language from becoming a bottleneck.

The escape hatch does not grant permission to change the surrounding theorem, import unrestricted axioms, or bypass the kernel. Its presence should be visible in the proof document, and its dependencies should be included in the same audit information as automatically generated

steps.

5 A proof contract and its intermediate representation

5.1 State and action nodes

A proof state can be modeled as

$$S = (\mathcal{E}, \Gamma, P, \mathcal{N}, \mathcal{M}, \mathcal{A}),$$

where $\mathcal{E}$ is the pinned declaration environment, Γ is an ordered dependent local context, P is the current proposition, $\mathcal{N}$ is the scoped notation environment, $\mathcal{M}$ is the set of enabled methods, and $\mathcal{A}$ is the assumption and execution policy.

A mathematical action produces a node containing its source range, its input state, the precise output proposition or reduced goals, and a proof constructor. For a simple reduction of P to $Q_1, \ldots, Q_m$, the essential object is a term of type

$$K : Q_1 \to \cdots \to Q_m \to P$$

in $\mathcal{E}; \Gamma$. More generally the premises form a dependent telescope, and some premises are functions over additional local variables. "Prove a statement for every point of an open set" cannot be represented as a hole at one arbitrary fixed point.

5.2 A contract has both logical and explanatory content

The *logical part* consists of types, local contexts, terms, and required premises. It determines whether a proof is valid. The *explanatory part* records why a method was chosen, which source phrase introduced the step, which facts were intended as inputs, and what the normalized expressions mean.

For example, a differentiation node can carry the following readable summary:

```
Action: differentiate the order-n identity at x
Input equality: iteratedDeriv(n,W) = G(n) o W on U
Locality obligation: U is a neighborhood of x
Derivative inputs: G(n) at W(x); W at x
Output: iteratedDeriv(n+1,W)(x) = G'(n)(W(x))*phi(W(x))
Evidence: neighborhood equality + derivative congruence + chain rule
```

This summary is generated from the typed node, not treated as a proof in its own right. It also prevents an implementation from describing one justification while silently proving the result by a completely unrelated theorem.

5.3 An obligation ledger

Every unresolved premise has a stable identifier, an exact local context, an origin, and a status. Useful statuses are *pending*, *checked*, *blocked by another obligation*, *unsupported by this method*, and *search exhausted*. An admitted conjecture may exist in a research notebook, but it must remain distinct from a checked theorem.

Automatically solved side conditions remain in the ledger, collapsed by default. The author can inspect a proof of $s \neq 0$, the reason a shifted coefficient vanishes, or the theorem certifying a change of representation. Hiding the proof text is a presentation decision. Removing the obligation would be a logical change.

5.4 Dependency graphs must respect scope

An obligation graph is more than a list of goals. Its edges record dependencies of types and terms, not merely the chronological order of search. A witness chosen inside one case cannot escape into the other case. A local assumption discharged by implication introduction cannot later be used as a global theorem. A metavariable cannot refer to a variable outside the context in which it was created.

The compiler therefore maintains dependent contexts and checks every assignment in that context. Acyclicity of the obligation graph is necessary for straightforward replay, but it is not sufficient: a graph can be acyclic and still contain an out-of-scope witness or a badly typed application. The kernel-level term construction is the final scope check.

5.5 A method library is a mathematical API

A method entry should specify a typed applicability test, its contract construction, its supported representations, and an explanation renderer. It should also expose its underlying theorem dependencies. A name such as `compare exponential coefficients` is useful only if it denotes an implementation of such an interface.

Method code can be untrusted as a producer of proof terms, provided its outputs are checked. Its *interpretation* still deserves review: a buggy method can suggest the wrong intermediate statement or provide a misleading explanation even when the final proof is logically valid. MPL should validate local step claims as well as the final theorem, so that the readable argument itself has a checked correspondence to its evidence.

6 Case study I: Abel coefficients without scalar bookkeeping

6.1 The exact mathematical setting

The source file defines Abel polynomials over a commutative ring and proves their exponential generating-function identities over a commutative $\mathbb{Q}$-algebra A. Write $\iota : \mathbb{Q} \to A$ for its specified scalar map. For $c \in A$, let

$$E_c(z) = \sum_{j \geq 0} \iota(1/j!) c^j z^j \in A[[z]].$$

The notation e^{cz} below abbreviates this *formal* series. It does not require an analytic exponential function on A.

Let $T \in A[[z]]$ satisfy the source's substitution equation

$$T = z\, E_{-a}(T). \tag{1}$$

The source first obtains the necessary zero constant coefficient of T, which licenses the substitution laws used subsequently. Its main coefficient theorem applies to *any* such T, not just the specially constructed series named `abelSeries`. For $n \in \mathbb{N}$, it establishes

$$[z^{n+1}]E_x(T) = \iota\left(\frac{1}{(n+1)!}\right) x\big(x - (n+1)a\big)^n. \tag{2}$$

Natural numbers multiplying elements of A in this display are interpreted through the specified algebra structure [1].

6.2 The mathematical proof

Put $m = n + 1$, so $m > 0$. The formal Lagrange–Bürmann coefficient theorem gives

$$\begin{aligned}
[z^m]E_x(T) &= \iota(1/m)[u^{m-1}](E_x'(u)E_{-a}(u)^m) \\
&= \iota(1/m)[u^{m-1}](xE_{x-ma}(u)) \\
&= \iota(1/m)\iota(1/(m-1)!)\, x(x-ma)^{m-1} \\
&= \iota(1/m!)\, x(x-ma)^{m-1}. \qquad (3)
\end{aligned}$$

The first line uses the selected inversion theorem and its substitution and inverse-series hypotheses. The second uses the derivative and product laws for formal exponentials. The third reads an exponential coefficient. The last is an identity of rational scalars transported into A.

The proof has a recognizable mathematical plan. The source implementation contains additional steps to produce `HasSubst T`, establish the substituted multiplicative inverse, instantiate the library theorem at $n + 1$, normalize the exponential parameter, and rearrange scalar maps and factorials. The explicit helpers `hcancel` and `hfactorial` are particularly clear examples of work that a reusable scalar-normalization method could absorb [1].

Here is the source's factorial helper, whose statement makes the rational-scalar transport explicit:

```
  have hfactorial :
      algebraMap ℚ A (1 / ((n + 1 : ℕ) : ℚ)) *
        algebraMap ℚ A (1 / n.factorial) =
      algebraMap ℚ A (1 / (n + 1).factorial) := by
    rw [← map_mul, one_div_mul_one_div, Nat.factorial_succ, Nat.cast_mul]
```

Listing 2: An actual excerpt from the Abel coefficient proof [1].

The proposed replacement is not to trust the phrase "factorials simplify." It is to package this family of map-and-scalar arguments once and reconstruct the relevant instance whenever the method is used.

6.3 A proposed proof block

In the context above, with target (2), the proof could be written:

```
proof
  put m := n + 1.
  have admissible: T has zero constant coefficient from equation.
  calculate coefficient(z^m, E(x,T))
    = (1/m) * coefficient(u^(m-1), D(E(x,u))*E(-a,u)^m)
        by Lagrange_Burmann using equation, admissible.
    = (x/m) * coefficient(u^(m-1), E(x-m*a,u))
        by formal_exponential_algebra.
    = (1/m!) * x * (x-m*a)^(m-1)
        by exponential_coefficients and rational_scalar_algebra.
  conclude after substituting m = n + 1.
end
```

Listing 3: An MPL reconstruction of the Abel coefficient argument.

Here every fraction outside a coefficient operation is elaborated as the action of a rational scalar. The notation pack cannot elaborate `x/m` by assuming that A is a field. The displayed proof is a design target for an adapter, not an executable script supplied by this article.

6.4 What the compiler has to reconstruct

The Lagrange step must resolve a specific theorem and produce its premises. In the inspected source, the inverse-series premise is justified by the formal identity

$$E_{-a}(T)E_a(T) = 1,$$

using admissible substitution and the corresponding identity before substitution. A theorem name is not sufficient evidence that these premises hold.

The scalar method must certify

$$\iota(1/m)\,(m\colon A) = 1, \qquad \iota(1/m)\,\iota(1/(m-1)!) = \iota(1/m!),$$

with $m > 0$. It can do so by proving the rational identities and applying the ring homomorphism laws. It need not prove that ι is injective, and it must not add a nontriviality assumption on A merely to use a preferred cancellation lemma. Even a degenerate coefficient algebra does not invalidate this scalar-identity route.

The normalizer must also distinguish the natural subtraction $m - 1$ from subtraction in A, and establish $m - 1 = n$ before rewriting the target. These facts are routine, but they still belong to the contract. A failed arithmetic side goal is a diagnostic, not a reason to reinterpret an index.

6.5 A meaningful second compression: the binomial identity

With

$$A_0(a, x) = 1, \qquad A_{n+1}(a, x) = x(x - (n + 1)a)^n,$$

the full generating-function identity includes the constant term:

$$E_x(T) = \sum_{n\geq 0} \iota(1/n!)A_n(a, x)z^n.$$

Multiplying the versions at x and y and using $E_x(T)E_y(T) = E_{x+y}(T)$ gives

$$A_n(a, x + y) = \sum_{k=0}^{n} \binom{n}{k} A_k(a, x)A_{n-k}(a, y).$$

The source proves the corresponding theorem via its exponential-generating-function and binomial-convolution interfaces [1]. A mathematical method can express this as "multiply the generating functions and compare exponential coefficients." The benefit comes from a reusable interface to these operations, not from hiding the defining identity of the polynomials in an unexplained call.

The zero coefficient deserves explicit attention. The positive-degree Lagrange formula does not itself prove the constant term. A method claiming the *full* series identity must generate and solve that additional obligation. This is an example of a detail that a human reader often supplies correctly but that an automatic compressor must not omit.

7 Case study II: differentiating an induction hypothesis locally

7.1 Preserve the source's weak hypotheses

Let $U \subseteq \mathbb{R}$ be open, and let $W, \varphi : \mathbb{R} \to \mathbb{R}$. Suppose

$$W'(x) = \varphi(W(x)) \quad (x \in U),$$

where the assumption means that the specified derivative *exists* at each such point. Let $G_n, H_n : \mathbb{R} \to \mathbb{R}$ be families such that, for every $n \in \mathbb{N}$ and $x \in U$,

$$G_0(W(x)) = W(x), \tag{4}$$

$$G_n'(W(x)) = H_n(W(x)), \tag{5}$$

$$G_{n+1}(W(x)) = H_n(W(x))\varphi(W(x)). \tag{6}$$

Again, (5) asserts the existence and value of the derivative. Then

$$W^{(n)}(x) = G_n(W(x)) \qquad (n \in \mathbb{N},\ x \in U). \tag{7}$$

This is the mathematical content of the source's main `iteratedDeriv_eq_comp` declaration [2]. The hypotheses concern G_n only at values in $W(U)$. Replacing them by global differentiability of every G_n would make a shorter statement easier to process, but would change the theorem. The source deliberately also offers a stronger-hypothesis wrapper; these are different interfaces, not interchangeable statements.

7.2 The proof and the missing neighborhood

For $n = 0$, (7) is (4). Assume the equality at order n for *all* $x \in U$. Fix such an x. Since U is open, the functions $W^{(n)}$ and $G_n \circ W$ agree on a neighborhood of x. Their derivatives at x therefore agree. By the chain rule and (5),

$$W^{(n+1)}(x) = H_n(W(x))\varphi(W(x)) = G_{n+1}(W(x)).$$

This completes the induction.

The source explicitly constructs an eventual equality in the neighborhood filter, then uses derivative congruence and `HasDerivAt.comp`. That is meaningful mathematical infrastructure, not a gratuitous technicality [2]. The language can infer its construction from the phrase "locally on the open set," but it cannot dispense with the premise.

7.3 A shorter narrative with a faithful lowering

Using the full context just stated:

```
proof
  induct on n with invariant
    forall x in U, iteratedDeriv(n,W)(x) = G(n)(W(x)).
  base:
    conclude by the zeroth derivative and G0.
  step n, assuming IH:
    take arbitrary x in U.
    differentiate IH locally at x on U
      using openness, derivatives_of_G, equation_for_W.
    conclude by the recurrence for G.
end
```

Listing 4: The induction invariant quantifies over the entire open set.

A domain adapter would lower the step through three recognizable facts: membership of U in the neighborhood filter of x, equality of derivatives of locally equal functions, and the chain rule. The order of tactic invocations need not appear in the surface text.

There is no claim that the original proof is dramatically longer. It is already concise and explains its local step. The point of the reconstruction is that the language names the precise mathematical operation and gives it a reusable contract. A fair implementation comparison would include an idiomatic Lean helper for exactly this local-differentiation pattern.

7.4 A negative example is part of the specification

At $x = 0$, the functions $f(x) = 0$ and $g(x) = x$ have the same value but different derivatives. Thus a command that infers $f'(0) = g'(0)$ from $f(0) = g(0)$ is unsound. This elementary counterexample should be a regression test for the differentiation adapter.

Similarly, an equality only on a closed or irregular set cannot automatically be handled by the open-neighborhood rule. A within-derivative theorem may apply under different hypotheses, but that is a different contract. The diagnostic should say that neighborhood equality was not established, rather than inventing smooth extensions or treating pointwise equality as function equality.

8 Case study III: an associated Stirling recurrence

8.1 Formal definition and generating functions

The source defines $S_r(n, k)$ using partial Bell polynomials with natural-number weights $[j \geq r]$. Its comments give the intended set-partition interpretation, but the present case study relies on the Bell-polynomial definition and the proved generating-function identities, not on an independently audited counting interpretation [3].

Fix $s \in \mathbb{N}$ and set $r = s + 1$. Work first in $\mathbb{Q}[[z]]$. Define

$$B(z) = e^z - \sum_{j=0}^{s} \frac{z^j}{j!}, \qquad C_k(z) = \sum_{n \geq 0} S_{s+1}(n, k) \frac{z^n}{n!}.$$

The column theorem and its elementary derivative consequence are

$$C_k = \frac{B^k}{k!}, \qquad B' = B + \frac{z^s}{s!}. \tag{8}$$

These are formal-series identities: no convergence theorem or exchange of analytic limits is needed.

8.2 Differentiate, normalize, compare

Differentiate the column at $k + 1$:

$$\begin{aligned} C'_{k+1} &= \frac{B^k}{k!} B' \\ &= \frac{B^{k+1}}{k!} + \frac{z^s}{s!} \frac{B^k}{k!} \\ &= (k+1) C_{k+1} + \frac{z^s}{s!} C_k. \end{aligned} \tag{9}$$

The coefficient of $z^n/n!$ on the left is $S_{s+1}(n + 1, k + 1)$. For the shifted term on the right it is

$$\begin{cases} \dfrac{n!}{s!(n-s)!} S_{s+1}(n - s, k), & s \leq n, \\ 0, & n < s. \end{cases} \tag{10}$$

Since $\binom{n}{s} = 0$ for $n < s$, the two cases combine into

$$S_{s+1}(n + 1, k + 1) = (k + 1) S_{s+1}(n, k + 1) + \binom{n}{s} S_{s+1}(n - s, k). \tag{11}$$

Here $n - s$ is natural-number subtraction. Its truncated value in the second case is harmless because the binomial factor vanishes.

This reproduces the mathematical content of `associatedStirling_succ_succ`. The source differentiates a power-series equality, applies the coefficient map, normalizes rational factors, explicitly splits the shifted-coefficient condition, and finally uses injectivity of the map from $\mathbb{N}$ to $\mathbb{Q}$ [3].

8.3 The proposed language exposes the right abstraction

A complete proof outline for the fixed natural-number target (11) is:

```
proof
  work on the equality in Q via the injective natural embedding.
  put B := exp(z) - sum(j=0..s, z^j/j!).
  put C(k) := egf(n |-> S(s+1,n,k)).
  have column: C(k) = B^k/k! by the Bell column theorem.
  have dB: D(B) = B + z^s/s! by the exponential tail identity.
  differentiate column at k+1 using dB.
  normalize to D(C(k+1)) = (k+1)*C(k+1) + z^s/s!*C(k).
  compare exponential coefficients at n:
    for s <= n, use the factorial formula for choose(n,s).
    for n < s, use zero support and choose(n,s) = 0.
  conclude in Nat by injectivity of the embedding.
end
```

Listing 5: The boundary convention is part of the coefficient method's contract.

The two explicit coefficient cases could later be folded into a certified shifted-EGF rule. Keeping them visible in the first implementation makes the method's boundary behavior easy to inspect and test.

8.4 A useful method theorem

Instead of hard-coding the Stirling recurrence, build a reusable coefficient theorem. For a sequence $a : \mathbb{N} \to \mathbb{Q}$, let

$$\mathrm{EGF}(a) = \sum_{n \geq 0} a_n \frac{z^n}{n!}.$$

For every $s, n \in \mathbb{N}$,

$$n![z^n] \left(\frac{z^s}{s!} \mathrm{EGF}(a) \right) = \binom{n}{s} a_{n-s}. \tag{12}$$

To prove it, split on $s \leq n$. In that case, coefficient shifting yields $a_{n-s}/(s!(n-s)!)$, and the factorial identity gives the result. In the other case, the shifted series has no coefficient at n, while the binomial coefficient is zero. This proof is short mathematically, reusable, and exactly where the index convention belongs.

Together with

$$n![z^n]\mathrm{D}\,\mathrm{EGF}(a) = a_{n+1},$$

linearity and scalar rules, (12) support a small certified coefficient compiler. The compiler need not solve arbitrary identities of series: it recognizes a restricted expression language and produces a theorem for each supported node. Unsupported expressions remain visible.

8.5 Three semantic mistakes that the language must prevent

First, applying the factorial quotient in (10) without $s \le n$ is not repaired by natural subtraction. The formula must have its support condition.

Second, the transport back to natural numbers uses *injectivity of the natural embedding into* $\mathbb{Q}$. It is not an unqualified rule allowing arbitrary rational expressions to be interpreted as natural-number expressions. Division and subtraction do not preserve their unrestricted meaning under such a move.

Third, the theorem is parameterized by $r = s + 1$, hence $r \ge 1$. A language that prints the result for "all r" has broadened the theorem beyond the reviewed bridge. The statement seal should expose this difference even if some separately proved $r = 0$ variant happens to exist.

9 Case study IV: an already compact scaling proof

The scaling file considers a specified polynomial map $F = (P, Q, R)$ and proves an identity of the form

$$F(x/s, sy, s^2 z) = \big(s^2 P(x, y, z),\ sQ(x, y, z),\ R(x, y, z)/s\big), \quad s \neq 0. \tag{13}$$

Its Lean proof uses function extensionality, the three coordinate cases, unfolding/simplification, and denominator normalization. The collision-preservation consequence is a short rewrite proof [4]. No broader conclusion about a conjecture is needed for this local example.

The source proof body itself is only:

```
funext i
fin_cases i <;>
  simp [evalMap, counterexample, scaleSource, scaleTarget] <;>
  field_simp
```

Listing 6: The existing scaling proof is already concise [4].

This is an important control: substantial algebra can already have a very short Lean proof. Replacing it by "obvious by algebra" would improve neither the mathematics nor the evidence. A useful new interface would instead offer a *certified weighted-homogeneity method*.

For a monomial $x^i y^j z^k$, assign source weight $-i + j + 2k$. Under the substitution in (13), it acquires the scalar factor $s^{-i+j+2k}$, interpreted in the field with $s \neq 0$. To establish target weights $(2, 1, -1)$, a checker can inspect the supported monomials of the three coordinates and verify their respective weights. It then uses a general theorem relating weighted homogeneity to scaling. A fallback method can simply verify all coordinates as the existing proof does.

A proposed surface block is therefore:

```
prove scaling by weighted homogeneity
  source weights (-1, 1, 2), target weights (2, 1, -1)
  using s != 0.
```

Its contract requires the weight checks or a coordinatewise identity certificate and the nonzero hypothesis. It does not assert the property merely because the requested weights look plausible. This adapter is a proposed implementation, not a claim that the source already uses a monomial-weight checker.

The benefit, where there is one, is explanatory reuse across a family of maps. The setup cost of proving the general weight theorem and implementing its checker must be counted in

any evaluation. On a single small example, the existing Lean proof may remain the better engineering choice.

10 Other mathematical phrases need equally precise contracts

10.1 Reindexing a finite sum

For finite index types I, J, an equivalence $e : I \simeq J$, and a function $f : J \to M$ into a commutative additive monoid, the equation

$$\sum_{i \in I} f(e(i)) = \sum_{j \in J} f(j)$$

follows by a certified reindexing theorem. A `reindex by e` method checks the domain, codomain, and inverse or bijectivity evidence. For bounded natural ranges, the proposed index map must also carry proofs of membership in the target range. The familiar reversal $i \mapsto n - i$ works on $\{0, \ldots, n\}$ with these bounds; it is not a global bijection of $\mathbb{N}$.

When a reindexing changes the summand as well, the method produces a separate pointwise summand-equality obligation. When it drops zero terms, it needs a support theorem. These obligations can often be synthesized by arithmetic and simplification, but they are not consequences of the word "reindex."

10.2 Without loss of generality

A `by symmetry` or `wlog` command should require a normalization principle. In one useful schema, a transformation $\sigma : X \to X$ and a region $R \subseteq X$ come with

$$\forall x,\ R(x) \vee R(\sigma x), \qquad \forall x,\ P(\sigma x) \to P(x).$$

To prove P everywhere it then suffices to prove it on R. The proof is a case split using coverage and the transport implication. A group action gives richer versions, but still needs orbit coverage and invariance information.

Thus "assume $a \leq b$ by symmetry" is valid only after the selected symmetry has been shown to preserve the relevant hypotheses and conclusion. A linear order may supply coverage, but it does not supply invariance of an asymmetric expression. The method should display which assumptions were transported.

10.3 Passing a limit, derivative, or integral through an operation

Formal power-series differentiation is an algebraic operator. Differentiating a function series term by term is an analytic theorem with convergence and regularity hypotheses. A shared word such as "differentiate" must not identify these operations. The expression's elaborated type and the explicitly selected domain determine the available method.

Likewise, "interchange sum and integral" should select a theorem whose exact hypotheses appear as obligations. The author may provide domination, nonnegativity, or another suitable condition, but the compiler should not guess which analytic theorem they meant merely because one leads to the desired formula. These cases motivate a method interface capable of presenting alternative contracts before committing to one.

10.4 Choosing an inverse or a witness

A proof of existence permits a local witness in the scope justified by existential elimination. It does not automatically define a globally computable function. Conversely, selecting an inverse

branch is often part of the mathematical specification, not a disposable proof detail.

MPL should let the author write "choose the positive solution" only when positivity and the relevant existence theorem are specified; uniqueness, when available, can justify a canonical choice. If several inequivalent witnesses satisfy the stated constraints, the chosen one should be exposed. A dependent pair carrying its specification is a better interface than an unexplained value selected by shortest-term ranking.

11 A small elaboration calculus

The surface language is open-ended, but its proof-producing core should be small. This section gives a mathematical specification of that core, not a mechanized implementation. It explains why additional mathematical methods can remain outside the logical trusted base.

11.1 Closed claims and scoped goals

Write $\mathcal{E}; \Gamma \vdash t : P$ for the underlying type-theoretic judgment. Here $\mathcal{E}$ supplies declarations and Γ is a well-formed dependent context. A public theorem has a fixed type $P_\star$ in its declared parameter context. All parameters, universe levels, and instances are part of this type; there are no unassigned statement metavariables when the theorem is sealed.

An internal goal may have more local variables. If a step needs to prove $Q(x)$ for every $x : A$, its obligation is not a proof of one selected $Q(x)$ but a term of type $\forall x : A,\, Q(x)$. Converting scoped goals into such explicit function types makes their dependencies visible at the boundary between the compiler and an external synthesizer.

11.2 Core constructors

For a nondependent logical subcore, the principal lowering rules are shown in Table 2. The fully dependent implementation uses the corresponding recursors and substitution operations. The table's variables are assumed to be well typed; it omits universe parameters and routine context-well-formedness premises.

Surface intention	Required premises	Constructed evidence
Conclude P	$t : P$	t
Assume $h : Q$ to prove R	$v : R$ in context $\Gamma, h : Q$	$\lambda h.v : Q \to R$
Establish $h : Q$, then P	$u : Q$ and $v : P$ under $h : Q$	$(\lambda h.v)u : P$
Suffices Q for P	$k : Q \to P$ and $u : Q$	$ku : P$
Take witness w	$w : A$ and $u : Q(w)$	$\langle w, u\rangle : \exists x : A,\, Q(x)$
Split $Q \vee R$	$h : Q \vee R$, $f : Q \to P$, $g : R \to P$	Disjunction elimination
Induct on n	$b : P(0)$ and $s : \forall n,\, P(n) \to P(n+1)$	Natural-number recursor
Use a method contract	$K : \Pi\vec{o}.\, P$ and checked arguments $\vec{p}$	$K\vec{p} : P$

Table 2: Mathematical surface forms lower to ordinary typed proof constructions.

The method row is the extension point. An external system may suggest K, the premises, and terms solving them. It has not extended the calculus: all of these objects must still be accepted in the underlying type theory.

For example, with hypotheses $h_1 : P \to Q$, $h_2 : Q \to R$, and $h : P$, a source step "conclude R from h through Q" lowers to $h_2(h_1 h)$. Its informative annotation identifies the intermediate

proposition Q, while type-directed synthesis supplies the applications. This is precisely the sort of structural task for which Leant is a natural candidate service [5].

11.3 Unfinished proofs are telescopes, not axioms

Suppose elaboration creates obligations $o_1, \ldots, o_m$. Each has a local context Δ_i and a target Q_i. Closing over its local context gives a type

$$\widehat{Q}_i = \Pi \Delta_i.\, Q_i.$$

Dependencies on earlier obligations are permitted when well scoped. The compiler can represent the unfinished proof as a typed skeleton

$$K : \Pi(o_1 : \widehat{Q}_1) \cdots (o_m : \widehat{Q}_m).\, P_\star. \tag{14}$$

The o_i are parameters of a function, not global declarations claiming that Q_i is true. Merely checking K does not prove $P_\star$. Completion requires arguments of the listed types, in dependency order.

In an implementation, Lean metavariables are convenient internal placeholders. At an export boundary, no unresolved metavariables or placeholder axioms remain. A draft notebook can store the skeleton and the outstanding goals, but its status is not that of a theorem.

Lemma 11.1 (Closing a dependent obligation telescope)**.** *Suppose* (14) *is well typed in a fixed environment and parameter context. For each i, suppose a term p_i has type $\widehat{Q}_i$ after substituting $p_1, \ldots, p_{i-1}$ for the earlier obligation parameters. Then $K p_1 \cdots p_m$ has type $P_\star$.*

Proof. Apply the dependent function-elimination rule to K and p_1. The result has the remaining telescope type with p_1 substituted for o_1. Apply the same rule to p_2, which has exactly the required substituted type. Continue inductively. After the final application, the result has type $P_\star$. The sealed theorem type has no free obligation parameters, so the conclusion is the original type, not a statement altered by the chosen solutions. □

11.4 Relative soundness of accepted elaboration

Proposition 11.2 (Relative proof preservation)**.** *Fix a well-formed declaration environment $\mathcal{E}$ and an explicit permitted-axiom policy $\mathcal{A}$. Assume that each accepted core node is translated by the corresponding typed constructor, that every method contract is itself checked, and that every obligation is closed by a checked term in its declared context. If export also verifies that the resulting term's type is the sealed theorem type $P_\star$ and that all transitive axiom dependencies satisfy $\mathcal{A}$, then an accepted* MPL *proof supplies an underlying proof of $P_\star$ relative to $\mathcal{E}$ and $\mathcal{A}$.*

Proof. Proceed by structural induction on the elaborated proof. The conclusion node supplies a term of the requested type. Assumption introduction uses function abstraction; the named-fact and sufficiency nodes use application; existential introduction and case analysis use their respective constructors and eliminators. Induction uses a recursor with the checked base and step premises. For each extensible method node, Lemma 11.1 closes its checked contract with the checked subproofs. Thus the root yields a well-typed term. The export check identifies its type with $P_\star$ under the declared conversion policy and verifies the permitted dependency condition. No unchecked search result is used as a proof premise. □

This is a relative proof-preservation argument, not a new proof of the consistency of Lean, a verified compiler implementation, or a guarantee that a human intended $P_\star$. The important engineering burden is to make the proposition's hypotheses true in the implemented pipeline. Checking only a final success flag would not do so.

11.5 Why this core is deliberately unambitious

The kernel should not contain a primitive rule called "the obvious coefficient argument" or "the usual compactness argument." Mathematical methods can evolve rapidly, but they should continue to emit ordinary proof terms. This permits aggressive search without making the search algorithm a logical authority.

Nor does the language need a complete procedure for all statements in its surface syntax. It can parse and correctly elaborate a statement whose proof remains beyond the enabled methods. A useful partial elaborator reports the exact unsolved proposition; an unsoundly accommodating one changes the proposition or supplies an unproved premise.

12 Statement fidelity is a separate verification problem

12.1 Proof validity does not settle meaning

Lean's validation documentation explicitly distinguishes a valid proof from the meaning of the theorem statement. It describes stronger workflows that compare a submitted proof with a separately established challenge statement and recheck exported evidence [9]. This distinction becomes more important, not less, when the surface language allows substantial inference.

Consider four superficially helpful changes: replace an arbitrary solution T by a canonical construction; assume every G_n is globally differentiable; restrict a recurrence to $s \leq n$; replace a commutative $\mathbb{Q}$-algebra by a field. Each can simplify a proof search. Each also changes one of the case-study statements. Kernel checking would establish the changed theorem, not detect the author's lost generality.

12.2 Seal the statement before searching for a proof

The proposed statement seal records the resolved theorem type, its parameter context, universe assignments, constants and definitions, selected instances, and notation provenance. The proof-search service receives this fixed target; it cannot modify it. A proof edit that requires a different target is presented as a statement edit with a semantic diff.

The seal has two representations. A canonical structural representation supports comparison and caching. A human-facing rendering shows the quantified variables, assumptions, domains, key definitions, and coercions. The former cannot make an ambiguous display clear by itself; the latter cannot replace structural comparison.

The initial seal should be produced in the trusted project environment, independently of an untrusted proof candidate. A malicious or mistaken candidate cannot be allowed to install syntax or type-class instances before the target is determined. Imported definitions must also be included in the reviewed semantic environment: matching only a theorem's printed name is insufficient.

12.3 Detect changes, but do not promise mind reading

A seal detects a change relative to a previously accepted interpretation. It does not prove that the original interpretation captured the author's intention. The author must still be able to inspect the resolved statement, follow notation to definitions, and check that the mathematical domains are right.

To reduce this burden, the interface can highlight material choices: formal versus analytic series, total versus restricted inverse, natural versus integer subtraction, and global versus local hypotheses. Such a "semantic choices" panel is a design proposal. It should report concrete

elaboration decisions, not a model-generated assurance that the theorem means what the author wanted.

12.4 Separate proof effects from object effects

For a fixed proposition P, different proof terms ordinarily do not change the theorem being stated. Data-valued choices can have quite different effects. A synthesized function $f : A \to B$, a point selected from a set, or a ring equivalence may appear in later statements and computations.

MPL should classify a synthesis request as proof-only, data-producing, or statement-resolving. The last category requires an explicit review boundary. Data-producing requests expose the chosen value and its specification. Definitionally identical alternatives may be merged safely; alternatives known equal by a checked theorem may be transported explicitly. Merely having the same type is not enough to treat two data terms as interchangeable.

Existence in `Prop` also does not automatically authorize arbitrary computational extraction into a data type. The implementation must respect Lean's elimination rules and declare any use of classical choice [10]. The friendly phrase "choose an object" should not conceal this foundational distinction.

13 An origin-bound synthesis protocol

13.1 Requests contain typed problems, not just printed goals

A proposed request format is:

```
SynthesisRequest
  environment_id, toolchain_id, method_versions
  statement_id, node_id, source_range
  local_context        // ordered, dependent, explicitly typed
  target               // elaborated expression and universes
  allowed_declarations, allowed_axioms
  effect_kind          // proof-only, data-producing, statement-resolving
  strategy_hint, search_budget, replay_budget
```

Listing 7: Conceptual protocol fields; this is not a current Leant API.

The identifiers bind records to a particular environment; they are not themselves proof evidence. The receiver validates the typed payload rather than trusting that a hash string denotes the correct context.

For an out-of-process backend, one route is to transmit a closed telescope type and a restricted declaration inventory. A native Lean adapter can instead preserve elaborated expressions directly within the appropriate environment. Neither route may forget local instance parameters, universe constraints, or dependency order while reducing a goal to a simpler search fragment.

13.2 A layered search policy

First use definitional reduction, explicit local facts, and small certified normalizers. These steps have low search ambiguity and often discharge coercion and arithmetic obligations. Next try structural synthesis and exact theorem instantiation. Then consult domain methods and a bounded tactic portfolio. More speculative search, including model-generated proof plans, belongs in an explicitly broader exploration mode.

This order is a proposed default, not a theorem about optimal search. A difficult goal may

benefit from a different strategy. The crucial constraint is that each strategy returns either checked evidence or a non-success result with its actual strength preserved.

Leant is a natural adapter for the structural layer. A formal-series adapter would handle coefficient and substitution contracts. A local-analysis adapter would handle derivative congruence and neighborhood constructions. A theorem index can help identify relevant declarations, but retrieval similarity is not a proof that a declaration has the right type.

13.3 Results preserve epistemic status

Useful result variants include:

```
Checked(term, exact_type, dependencies, replay_information)
Candidates(list_of_unaccepted_suggestions)
Unsupported(reason, unsupported_structure)
Exhausted(budget_used, frontier_summary)
RefutationEvidence(fragment, translation_evidence, certificate)
```

The last variant is not automatically a Lean proof of the negated target. The adapter must establish what is refuted, whether the source-to-fragment translation preserves the relevant semantics, and whether its evidence can be lifted to the original statement. Otherwise it remains a separately labeled diagnostic.

An external solver's `sat`, `unsat`, or `unknown` message is not an additional result constructor with theorem authority. A model may guide search or yield a concretely checkable counterexample. A proof certificate may support reconstruction. Without the relevant checked correspondence, the result cannot close a goal.

13.4 Bind acceptance to the exact candidate

A candidate returned as text is re-elaborated in the exact original context. Its resulting term and type are retained. A ranking stage may reorder candidates, but it must not detach a proof receipt from the candidate for which it was produced. A rewrite, wrapper, or changed instantiation creates a new candidate requiring validation.

This directly reflects the useful boundary discipline in Leant's semantic-origin and selection design [6, 7]. MPL extends the association to the narrative node: the step's displayed result, its contract, and its accepted evidence remain connected. In particular, two identical printed strings can resolve differently under different local contexts or instances.

13.5 Ranking should optimize useful proofs, not only short terms

A possible ranking objective is a vector rather than a single term-length score:

$$(\text{unrequested semantic choices},\ \text{new premises},\ \text{explanatory mismatch},\ \text{replay cost},\ \text{term size}).$$

Unpermitted assumptions are a rejection condition, not merely a penalty. The other dimensions can guide ordering among valid alternatives. For data terms, satisfying an explicit behavioral specification outranks superficial compactness. For proof terms, a direct use of the author's cited lemma may be preferable to a large unrelated argument.

These quantities are design targets and need practical approximations. No claim is made that they define a computable optimum over all proofs. The important point is that shortest syntax is not a reliable proxy for the clearest explanation or the intended construction.

13.6 Suggestions are transactions

A tactic suggestion should be tested in an isolated snapshot of the current proof state. The interface can distinguish a suggestion that merely makes progress from one that actually closes the exact goal. Accepting it commits its validated state transition and evidence; rejecting it restores the previous state.

Search should not silently install declarations, alter a global simplification set, or leave changed metavariable assignments in neighboring goals. Branches may share immutable library data, but their speculative contexts are isolated. The same rule applies when a model proposes an intermediate lemma: the lemma becomes an accepted premise only after its proof is checked.

14 Trust, reproducibility, and hostile inputs

14.1 A proof artifact is more than a successful build

An accepted artifact should contain the sealed statement, the proof term or reproducible generated Lean source, the mathematical action graph, and the dependency and assumption inventory. It should record the relevant toolchain and library revisions and retain enough information to map each source step to its evidence.

The proof term is the central mathematical artifact. A search transcript is useful provenance, but is not needed as an authority once a valid term is available. Conversely, a transcript saying "all goals solved" is not a substitute for that term. A cache hit still undergoes type and policy checks before it contributes evidence to a new build.

14.2 Make the foundational policy explicit

Lean permits axioms, and its documentation identifies standard foundational axioms and explains how to inspect transitive dependencies [10]. MPL should allow projects to declare a foundational policy rather than demand an indiscriminate "no axioms" condition. A classical-mathematics project and a constructive-development project may make different choices.

The strict publication policy should reject placeholder axioms and undeclared mathematical assumptions. A theorem deliberately conditional on a conjecture belongs to a visibly conditional section, with the assumption present in its statement or dependency report. It must not inherit the same status as an unconditional checked theorem merely because the kernel can reason from the declared axiom.

Native computation requires equally careful accounting. Current Lean documentation explains that the dependency mechanism for native evaluation has changed across versions, including dedicated computation axioms in newer versions [9]. An audit should inspect the actual pinned toolchain's dependency inventory, not search only for one historical axiom name. A strict kernel-only policy should reject unapproved native-computation assumptions; a broader policy should display its larger trust base.

14.3 Untrusted elaboration is also executable code

A proof-generating tactic is not logically trusted merely because it runs inside Lean, but it may still execute code with access to the host system. Consequently, an untrusted proof submission needs execution isolation as well as proof checking. This threat model is explicitly recognized in Lean's validation guidance [9].

The proposed architecture separates an untrusted construction process from a clean validation process. It bounds external-process resources, restricts file and network access, validates serial-

ized evidence, and compares the theorem against the sealed statement outside the construction environment. Imported method code receives no special exemption. A kernel check performed after an untrusted process has compromised its own checking environment is not the intended security boundary.

These requirements are architectural specifications, not a claim that a particular sandbox is infallible. The remaining assumptions include the logic, checker implementations, serialization boundary, operating environment, and the correctness of the reviewed statement.

14.4 Replay and library evolution

The accepted proof should be replayable without repeating stochastic or expensive discovery. Discovery can return several candidates; publication retains one checked realization plus the mathematical source. A replay path may use stored terms or deterministic generated proof code, with all relevant dependencies pinned.

After a library upgrade, the system first tests whether the old artifact still checks. If a method must be rerun, its new result is compared against the sealed statement and the old action contracts. A changed theorem hypothesis or instance selection requires a visible semantic diff. A different proof term of the same theorem may be acceptable, but changes in declared premises and foundational dependencies remain reviewable.

The practical cache key includes the target, local context, referenced declarations, notation and method versions, and relevant policy. Even a perfect key is only an optimization: it does not authorize accepting a malformed or stale term. Validation of the exact resulting artifact remains mandatory.

15 The reading and authoring experience

15.1 One proof, three levels of detail

The reader's default view should contain the theorem, the mathematical plan, the substantial intermediate claims, and the named methods. Expanding a line reveals the contract and its side conditions. A further expansion reveals the exact Lean evidence. The views are projections of one typed document, not three separately maintained explanations.

For the Stirling example, the compact view might show only "compare exponential coefficients." The next level shows the cases $s \leq n$ and $n < s$, the factorial normalization, and the embedding into $\mathbb{Q}$. The lowest level shows the particular coefficient and cast lemmas. This hierarchy lets a mathematician read at the level of the proof's idea without depriving a reviewer of the details needed to inspect it.

An explanatory sentence generated from the action graph should accurately describe the evidence actually used. A free-form commentary paragraph can remain unverified exposition, but the interface must distinguish it from a checked proof step. It should not print a convincing explanation next to unrelated evidence and present their correspondence as certified.

15.2 Diagnostics should report mathematical obstacles

A useful failed Abel step says that the chosen substitution law requires admissibility of the inner series, and points to the missing constant-coefficient or admissibility fact. A failed derivative step says that equality near the selected point has not been established. A failed transport says that equality reflection is missing for the selected map.

Such messages are produced from contract premises. The system can still expose the raw

Lean goal, but it should not force the author to infer the mathematical issue from an internal expression mismatch alone. A diagnostic should distinguish an actually false premise, an unsupported method, and a search budget exhaustion. Only evidence can justify the first classification.

15.3 Research notebooks and publication mode

During exploration, a notebook may contain conjectures, attempted lemmas, unchecked candidate steps, and checked results. The user may ask a synthesizer to fill a hole or a model to propose an invariant. These are useful activities, but their statuses should remain explicit.

Publication mode requires that every dependency of the advertised theorem has the required checked or explicitly assumed status. It freezes the resolved statements and accepted evidence, refuses unresolved holes, and generates a source-facing proof document. An unresolved conjecture elsewhere in the notebook need not block an independent theorem, provided the dependency graph establishes that independence.

This separation is especially useful for a repository containing both formal developments and research exposition. A mathematically suggestive paragraph does not acquire formal status merely because nearby theorems are checked. The document should make proof-backed claims and research proposals easy to distinguish.

15.4 The author's role changes, but does not disappear

The proposed interface moves effort from tactical bookkeeping toward choosing definitions, specifying the theorem, identifying a useful invariant, and selecting mathematically meaningful intermediate statements. It cannot remove those decisions. Indeed, stronger automation makes a precise specification more important because an underspecified type may have many convenient but unintended solutions.

The intended reading experience resembles a mathematical proof, not a transcript of the prover's search. The search may try dozens of unhelpful rewrites or candidate terms; the accepted document records the successful argument and its dependencies. The exploration history remains available separately when it is useful for research provenance.

16 Implementation strategy: extend Lean, do not replace its kernel

16.1 Begin with a Lean-hosted front end

The most direct prototype is a Lean extension with a small syntax layer, elaborators for scoped blocks, and a typed intermediate representation. Formula syntax should initially reuse Lean's term language plus carefully specified notation packs. This preserves interoperability with existing definitions and avoids reimplementing the dependent type system.

The first milestone is deliberately narrow: `have`, `suffices`, `calculate`, `induct`, explicit local contexts, and a Lean escape hatch. It should export ordinary Lean declarations and a readable contract report. Before adding broad automation, it must correctly reject unsolved obligations and scope violations.

16.2 Implement two mathematical method libraries first

A formal-series library should support typed coefficient extraction, exponential-generating-function normalization, scalar maps from $\mathbb{Q}$, support-aware shifts, and a bridge to an existing Lagrange theorem. It should expose which laws require substitution admissibility and which

manipulations are purely algebraic.

A local-calculus library should support passing from equality on an open set to equality near a point, applying derivative congruence, and assembling chain-rule premises. It must preserve range-local assumptions rather than automatically selecting a stronger globally differentiable theorem.

These two libraries cover the first three case studies and create reusable infrastructure. The weighted-scaling method is a useful later addition and a control against overengineering. Existing Lean lemmas and tactics should be reused wherever possible; the new code primarily organizes their mathematical contracts.

16.3 Integrate Leant behind an exact-goal adapter

The adapter should first export the current Lean goal and context through an explicit capability-checked translation. Leant can propose structural inhabitants or tactic suggestions; the host accepts only candidates validated against the original problem. The protocol in Section 13 is a target interface, not an assumption that Leant's current external API already exposes every required field.

A native Lean implementation of some synthesis components could reduce the cost of preserving elaborated expressions and typed environments. An external Haskell service can still be useful. The architectural requirement is the same in both cases: the smaller search representation must not become the authority for the full theorem's semantics.

An unsupported dependent feature should lead to a narrower search request, another backend, or an explicit local proof. It should not be erased merely to fit the synthesizer's fragment. Positive candidate rechecking can protect against many translation mistakes; negative results need a stronger completeness and correspondence story.

16.4 Freeze evidence before adding ambitious natural language

The next milestone is stable replay: accepted steps retain proof terms or deterministic proof realizations, their exact statements, and their method dependencies. A statement-diff interface and dependency audit should work before a general language model is allowed to propose large proof blocks.

Only then add a natural-language editing assistant. Its output is a patch to the mathematical source, with changes to theorem statements and data choices separately highlighted. The assistant does not create a parallel channel that can accept untyped text as evidence. This order of development makes the system useful even when model-generated plans are poor.

16.5 An implementable method boundary

A conceptual method interface is:

```
prepare : TypedState -> ParsedAction ->
          Either Diagnostic ProposedContract

validateContract : ProposedContract ->
          Either Diagnostic CheckedContract

solvePremise : CheckedContract -> PremiseId ->
          SearchResult

assemble : CheckedContract -> CheckedPremiseSolutions ->
          Either Diagnostic CheckedTerm
```

The names denote responsibilities, not a complete implementation. A caller cannot create a checked contract merely by supplying a boolean flag. Its construction must check the contract's dependent type. `assemble` checks the resulting term and its target again. Explanatory metadata travels beside these objects but cannot supply a missing premise.

17 How to evaluate the proposal fairly

17.1 Compare against good Lean, not an artificially weak baseline

The primary baseline should be idiomatic, refactored Lean using the same mathematical helper theorems and comparable automation. Comparing a one-line MPL method call with a raw Lean proof that reimplements the method would mostly measure library engineering, not language design.

A useful study has four conditions: the existing source, an idiomatic Lean refactoring, a notation-and-structure-only MPL layer, and the full contract-and-synthesis system. The comparison between the second and fourth conditions is especially important. All conditions must preserve the same elaborated theorem and foundational policy.

The four inspected samples are suitable pilot tasks, not enough for a broad claim. A larger corpus should include proof families from algebra, combinatorics, analysis, and dependent constructions, with held-out tasks that were not used to design the method libraries.

17.2 Measure three different costs

Authoring cost includes the theorem statement, proof body, project-specific configuration, new helper lemmas, and method-pack development. Report both the one-time setup cost and its amortization across a family of proofs.

Comprehension and repair cost includes understanding a proof's strategy, locating a missing hypothesis, predicting the meaning of a transport, and repairing a proof after a library or statement change. A shorter file does not necessarily reduce these costs. User studies should control for participants' Lean experience and familiarity with the mathematics.

Machine cost includes discovery time, replay time, peak memory, proof-term size, search failures, and dependency growth. A concise source can produce a very large certificate. Such tradeoffs should be reported rather than hidden behind a source-token count.

All measurements proposed here remain to be performed. The worked reconstructions demonstrate a target interface and mathematical correspondence; they do not establish a numerical improvement.

17.3 Negative tests are first-class acceptance criteria

Table 3: Suggested regression tests for mathematical and operational fidelity.

Mutation or attack	Required behavior
Drop openness in the derivative proof	Do not use the open-neighborhood rule without replacement evidence; expose the missing locality premise.
Use equality at one point	Reject derivative congruence unless a suitable local equality has been established.
Apply a factorial quotient when $n < s$	Generate the zero-support branch rather than silently imposing $s \leq n$.
Remove $s \neq 0$ from a scaling method	Require denominator or unit evidence; do not assume cancellation.
Replace a general $\mathbb{Q}$-algebra by a field	Mark a theorem-statement change, even if proof search becomes easier.
Replace arbitrary T by a canonical solution	Reject it as a realization of the old sealed statement unless an exact bridge proves the original claim.
Reuse a witness from another case	Reject the out-of-scope term, even if the obligation graph is acyclic.
Return a type-correct but wrong-behavior function	Do not claim its behavioral specification without a checked proof; expose a data choice.
Exhaust a bounded synthesizer	Report exhaustion, not refutation or falsehood.
Transfer a receipt after changing a candidate	Revalidate the changed term and its association with the original problem.
Hide an admitted dependency	Reject publication under a policy excluding placeholder or undeclared axioms.
Change notation or an instance under the same printed goal	Compare elaborated statements and environments; invalidate stale evidence associations.
Use a circular auxiliary lemma	Keep both goals unresolved; do not publish a cyclic collection of assumed statements.

These tests should be automated where possible and supplemented by human review of the diagnostic text. Passing a logical rejection test does not show that the explanation is comprehensible, and a helpful explanation does not show that the rejected artifact could not bypass another validation path.

17.4 A falsifiable success criterion

The proposal succeeds if authors can state and maintain the same theorems with less representational work, while readers can recover the strategy and inspect all material hypotheses, at acceptable checking cost. It fails in an important sense if it merely moves a large amount of one-off code behind pleasant keywords, hides semantic strengthening, or makes failed proofs harder to diagnose.

A partial success is still useful: a small stable library for coefficient arguments may justify itself

even if general natural-language authoring remains unreliable. The evaluation should allow such domain-specific conclusions instead of forcing a universal claim about "human-like proofs."

18 Relationship to existing work

18.1 Declarative proof documents

Isar provides an established model of intelligible structured proofs and distinguishes document-level proof structure from exploratory interaction [11]. MPL should borrow that discipline. Structured blocks, named facts, and readable proof documents are not new contributions of this proposal.

Isabelle/Naproche accepts controlled mathematical language in ForTheL, including a LaTeX dialect, and uses automated reasoning to fill implicit steps [12]. It is a particularly relevant predecessor for the goal of readable mathematical documents. The cited paper also distinguishes integration with Isabelle's document infrastructure from a logical connection to Isabelle/HOL. A familiar editor or syntax is therefore not, by itself, a statement about the proof-checking foundation.

18.2 Controlled natural language in education

Waterproof was developed to help students write mathematical proofs in a controlled language resembling traditional proof text, with an associated editing environment [13]. Verbose Lean uses Lean's extensibility to support paper-like proof writing with configurable pedagogical assistance [14]. These systems already demonstrate that one need not choose between conventional tactic scripts and wholly unconstrained prose.

The present proposal has a different emphasis: research-level representation changes, explicitly typed method contracts, exact statement preservation across synthesis, and reproducible evidence attached to the narrative. This is a proposed combination and engineering agenda, not a claim to have invented natural-language proofs or to supersede the educational goals of those systems.

18.3 Automation and Lean's existing abstraction mechanisms

Aesop is a Lean proof-search system organized around configurable rules, search, and normalization [15]. It is a potential backend for suitable obligations, not a competing mathematical surface language. Leant contributes type-directed structural synthesis and an explicit discipline around exact-goal validation and evidence associations [5, 6].

Lean itself already offers elaboration, notation, macros, structured proof constructs, and metaprogramming [8]. Many individual MPL features can therefore be implemented without a new language name at all. The argument for a coherent layer is that separately available mechanisms do not automatically provide a shared statement seal, a typed action ledger, a consistent synthesis protocol, and a multi-resolution mathematical reading view.

18.4 What is and is not being claimed as new

The potentially useful contribution is the joint design: mathematical-action contracts serve simultaneously as elaboration interfaces, explanatory units, scope boundaries for synthesis, and stable links to final evidence. Leant-inspired origin binding is extended from candidates to the relation between the source statement, each mathematical step, and its accepted proof realization.

No exhaustive novelty search has been performed, and no priority claim is made for the individual

ingredients. The proposal should be judged by whether this integration can be built and whether the evaluation in Section 17 demonstrates an advantage over carefully engineered Lean.

19 Limitations and conclusions

Several difficulties remain substantial. General proof search is not made complete by friendly syntax. Mathematical notation is contextual. Useful method libraries require real formalization effort. A proof that depends on a novel argument cannot be replaced by a keyword until that argument has been supplied or discovered. A compact source may still yield expensive evidence, and a formally valid theorem can still be an incorrect formalization of the intended claim.

The proposed response is not to weaken checking, but to divide responsibility more cleanly. The author supplies the mathematical objects, claims, and strategic choices. The language gives these choices precise scoped meanings. Domain methods reconstruct admissible representation changes and side conditions. Synthesizers provide candidate terms and local proof completions. A validation boundary checks the resulting evidence against an independently fixed statement and an explicit foundational policy.

The ProveIt examples show why this division is promising. The Abel coefficient argument has a short mathematical calculation whose scalar bookkeeping can be factored out. The autonomous-derivative proof has a local reasoning step whose neighborhood premise must survive compression. The associated Stirling recurrence needs a reusable support-aware coefficient calculus, not an omitted boundary case. The scaling proof reminds us that existing Lean can already be very concise.

The design principle can be stated simply:

> **Make mathematical intention the unit of authorship, and make fully checked evidence the unit of acceptance.**

A successful implementation would not ask mathematicians to abandon rigor or require them to narrate every kernel step. It would let them write the reasons that matter, while ensuring that every omitted detail is either recovered with evidence or returned as an explicit mathematical question.

A A compact grammar and boundary conventions

The following grammar describes a possible structural core. It deliberately leaves mathematical formulas to a typed expression parser and specialized actions to registered method grammars. It is not a complete parser specification for every illustrative phrase in this article.

```
proof       ::= "proof" step* "end"
step        ::= "have" label ":" formula justification
              | "suffices" formula "by" justification
              | "assume" binder proof
              | "take witness" term justification
              | "cases" term branch+
              | "induct" term "with invariant" formula branch+
              | "calculate" calculation
              | "conclude" [formula] justification
              | method_action
              | "lean" lean_block
justification ::= "from" labels ["using" method]
                | "by" method ["using" labels]
```

```
                | proof
calculation ::= expression relation_edge+
relation_edge ::= relation expression justification
```

Names are lexically scoped. Formulas are resolved before proof search is permitted to act on them. Overloaded symbols are accepted only when their types and scoped notation determine a unique intended interpretation under the declared policy; otherwise a source annotation is required. Definitional conversion may justify equal elaborations, but proof search is not used to select whichever theorem statement happens to be easiest.

A `method_action` parses into a named adapter and a typed argument request. The adapter may propose alternative explicit contracts when the action is underspecified. Accepting one records the choice. It may not silently add a premise to the theorem or reinterpret a mathematical object.

Free prose belongs to marked exposition or to the editing-assistant channel. It can explain motivation, failed attempts, or conjectural ideas, but only recognized and checked proof nodes count toward the theorem's evidence.

B Source register and reproducibility notes

B.1 ProveIt source crosswalk

All four references use revision `24ce8bd743eaab64a91ce90725ea00f498d319d2`. The relevant paths and declarations are:

Abel coefficients. Under `Analysis/FabiusFunction/Lean/FabiusFunction/AbelPolynomialSeries.lean`: `coeff_exp_subst_of_abel_eq`, `abel_eq_zero_and_one`, `exp_subst_eq_egfA_abelPolynomial`, and `abelPolynomial_eval_add` [1].

Autonomous derivatives. Under `Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean`: `AutonomousODE.iteratedDeriv_eq_comp` and the stronger-hypothesis wrapper `iteratedDeriv_eq_comp_of_differentiable` [2].

Associated Stirling numbers. Under `Analysis/FabiusFunction/Lean/FabiusFunction/AssociatedStirling.lean`: `egfA_associatedStirling`, `derivative_bellWeightSeries_assocWeight`, and `associatedStirling_succ_succ` [3].

Scaling. Under `Algebra/JacobianConjecture/Lean/JacobianConjecture/Scaling.lean`: `counterexample_scaling` and `collision_scaling` [4].

The article discusses the mathematical content of these declarations and their local proof patterns. It does not make claims about unrelated declarations or about the repository's overall proof completeness.

B.2 Leant inspection boundaries

The README sections on interactive proving, automatic synthesis, structural fragments, negative verdicts, and behaviorally different inhabitants were inspected at revision `3a40904be8a410d832d9ab6900b3d3e7b425eccb`. The public wrapper `src/Leant/Synth/BehavioralSelection.hs` was inspected at that revision as well [5, 7].

The semantic-origin, provider-binding, candidate-association, and Length handoff discussion in `docs/synth-internals.md` was inspected at revision

`c36adf114035fb2fba6c276b63944cc613b31668`

as recorded in reference [6]. This is a source-and-documentation review of selected boundaries, not a complete audit of the synthesis engine or an independent rerun of the repository's reported acceptance tests.

B.3 Evidence and status ledger

Category	Status in this article
Repository observations	Based on the cited source files and documentation at the stated revisions.
Mathematical reconstructions	Explicit derivations in Sections 6–9; not newly executed formal proofs.
Language and compiler design	Proposed syntax, contracts, protocol, and implementation strategy.
Relative soundness	Mathematical argument for the stated core and assumptions; not a mechanized compiler theorem.
Performance and usability	Evaluation plan only; no measured outcomes or savings claimed.
Artifact validation	The accompanying LaTeX source is compiled to the supplied PDF; this is document validation, not Lean verification.

B.4 Building the document

The article is self-contained in `article.tex`; the bibliography is embedded, and no external figures or font files are required. A standard TeX Live installation with the listed packages can build it with

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

or by running `pdflatex` enough times to resolve the table of contents and references. The distributed archive includes the PDF and a short README. Its proof-language listings are illustrative and are not compilation inputs for Lean.

References

[1] Vladimir Reshetnikov and contributors. *ProveIt: AbelPolynomialSeries.lean*. Revision 24ce8bd743eaab64a91ce90725ea00f498d319d2. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AbelPolynomialSeries.lean.

[2] Vladimir Reshetnikov and contributors. *ProveIt: AutonomousIteratedDeriv.lean*. Same ProveIt revision. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean.

[3] Vladimir Reshetnikov and contributors. *ProveIt: AssociatedStirling.lean*. Same ProveIt revision. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AssociatedStirling.lean.

[4] Vladimir Reshetnikov and contributors. *ProveIt: Scaling.lean*. Same ProveIt revision. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Algebra/JacobianConjecture/Lean/JacobianConjecture/Scaling.lean.

[5] Vladimir Reshetnikov and contributors. *Leant: README and synthesis/proving guide*. Revision 3a40904be8a410d832d9ab6900b3d3e7b425eccb. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/README.md.

[6] Vladimir Reshetnikov and contributors. *Leant: :synth internals*. Revision c36adf114035fb2fba6c276b63944cc613b31668. https://github.com/VladimirReshetnikov/Leant/blob/c36adf114035fb2fba6c276b63944cc613b31668/docs/synth-internals.md.

[7] Vladimir Reshetnikov and contributors. *Leant: BehavioralSelection.hs*. Revision 3a40904be8a410d832d9ab6900b3d3e7b425eccb. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/BehavioralSelection.hs.

[8] Lean developers. *The Lean Language Reference: Elaboration and Compilation*. Online reference, accessed September 5, 2026 (Pacific time). https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/.

[9] Lean developers. *The Lean Language Reference: Validating a Lean Proof*. Online reference, accessed September 5, 2026 (Pacific time). https://lean-lang.org/doc/reference/latest/ValidatingProofs/.

[10] Lean developers. *The Lean Language Reference: Axioms*. Online reference, accessed September 5, 2026 (Pacific time). https://lean-lang.org/doc/reference/latest/Axioms/.

[11] Isabelle project. *Isar—Intelligible semi-automated reasoning*. Project description and bibliography. https://isabelle.in.tum.de/Isar/.

[12] Adrian De Lon, Peter Koepke, Anton Lorenzen, Adrian Marti, Marcel Schütz, and Makarius Wenzel. *The Isabelle/Naproche Natural Language Proof Assistant*. In *Automated Deduction—CADE 28*, LNCS 12699, pp. 614–624, 2021. https://doi.org/10.1007/978-3-030-79876-5_36.

[13] Jelle Wemmenhove, Dick Arends, Thijs Beurskens, Maitreyee Bhaid, Sean McCarren, Jan Moraal, Diego Rivera Garrido, David Tuin, Malcolm Vassallo, Pieter Wils, and Jim Portegies. *Waterproof: Educational Software for Learning How to Write Mathematical Proofs*. arXiv:2211.13513, 2022. https://arxiv.org/abs/2211.13513.

[14] Patrick Massot. *Teaching Mathematics Using Lean and Controlled Natural Language*. In *ITP 2024*, LIPIcs 309, pp. 27:1–27:19, 2024. https://doi.org/10.4230/LIPIcs.ITP.2024.27.

[15] Jannis Limperg and Asta Halkjær From. *Aesop: White-Box Best-First Proof Search for Lean*. In *CPP 2023*, pp. 253–266, 2023. https://doi.org/10.1145/3573105.3575671.